

LOW LEVEL DESIGN (LLD)

Mutual Fund Purchase Optimizer System

Table of Contents

1. Introduction
 2. Design Objectives
 3. Architectural Alignment with HLD
 4. Technology Stack
 5. Presentation Layer Design
 6. Infrastructure Layer Design
 7. API Layer Design
 8. Application Layer Design
 9. Recommendation Engine Design
 10. Repository Layer Design
 11. Data Layer Design
 12. Error Handling Strategy
 13. Security Design
 14. Performance & Scalability Design
 15. Deployment Design
 16. Sequence Flow – Recommendation Generation
 17. Updated LLD Architecture Diagram
 18. Assumptions & Constraints
 19. Traceability to HLD
 20. Conclusion
-

1. Introduction

1.1 Purpose

This Low Level Design (LLD) document provides the detailed technical architecture for implementing the **Mutual Fund Purchase Optimizer System**.

The system generates optimized mutual fund purchase recommendations based on:

- investment amount
- investment duration
- client risk profile
- existing portfolio holdings

This document translates the **High-Level Design (HLD)** into detailed modules, components, workflows, and database structures required for implementation.

2. Design Objectives

The LLD aims to achieve the following objectives:

- Maintain strict alignment with the HLD layered architecture
- Support deterministic recommendation logic
- Provide explainable investment recommendations
- Achieve response time under **2 seconds**
- Support scalability and cloud deployment
- Maintain secure and auditable workflows

3. Architectural Alignment with HLD

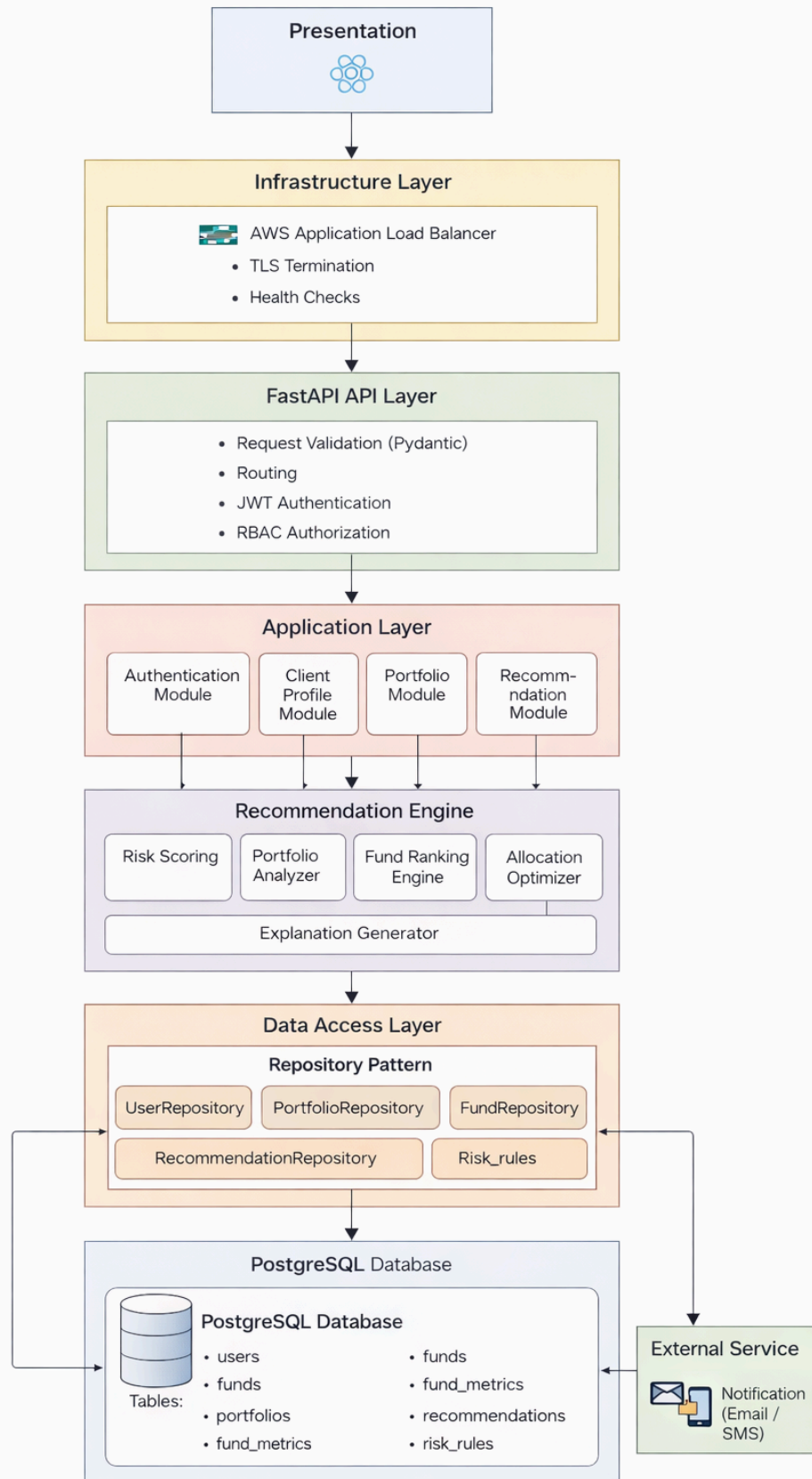
The system follows a **layered architecture** as defined in the HLD.

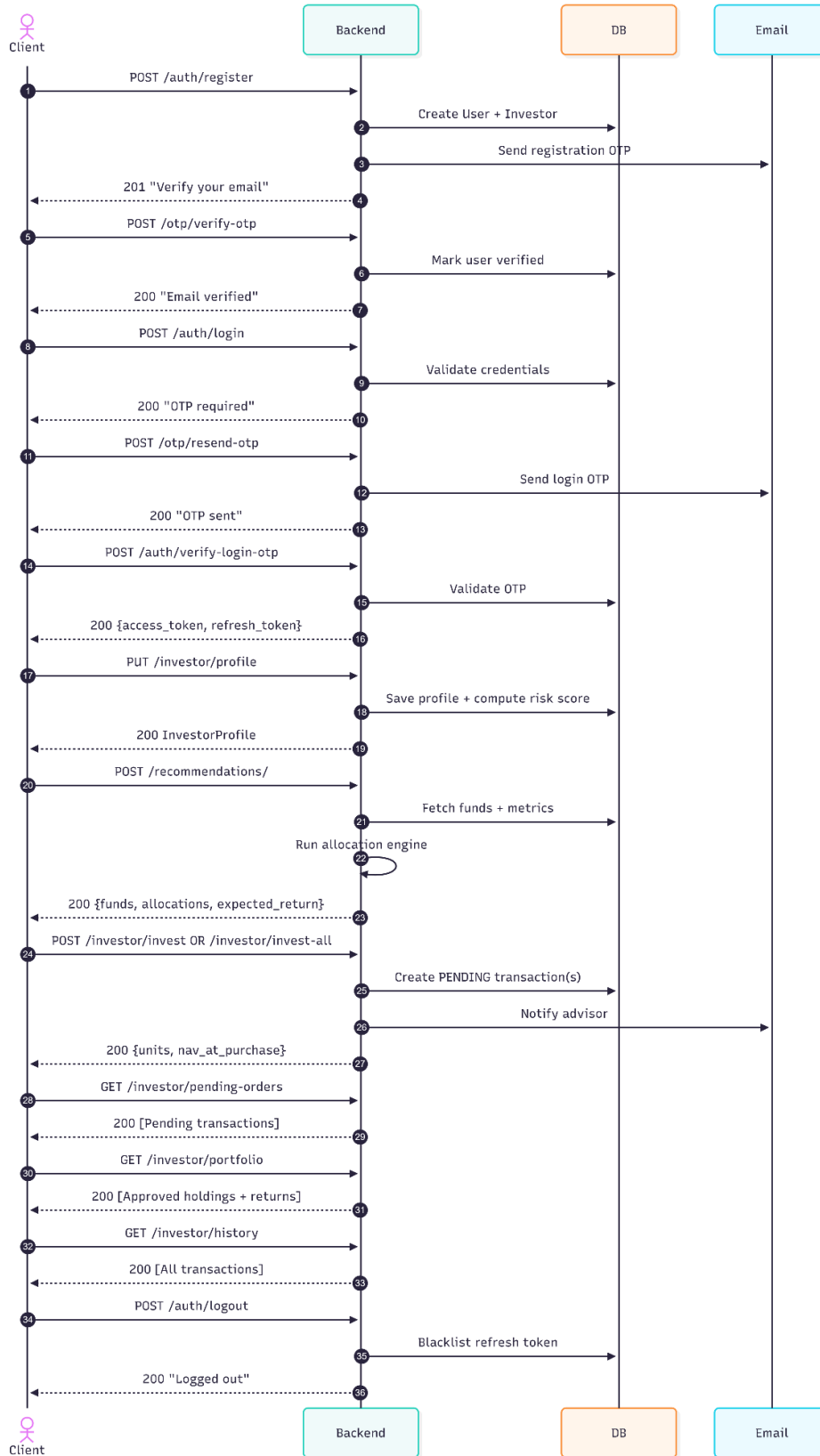
HLD Layer	LLD Implementation
Presentation Layer	React Web Application
Infrastructure Layer	AWS Application Load Balancer
API Layer	FastAPI Application
Application Layer	Business Modules
Recommendation Engine	Optimization Components
Data Access Layer	Repository Pattern
Data Layer	PostgreSQL Database
External Systems	Notification Service

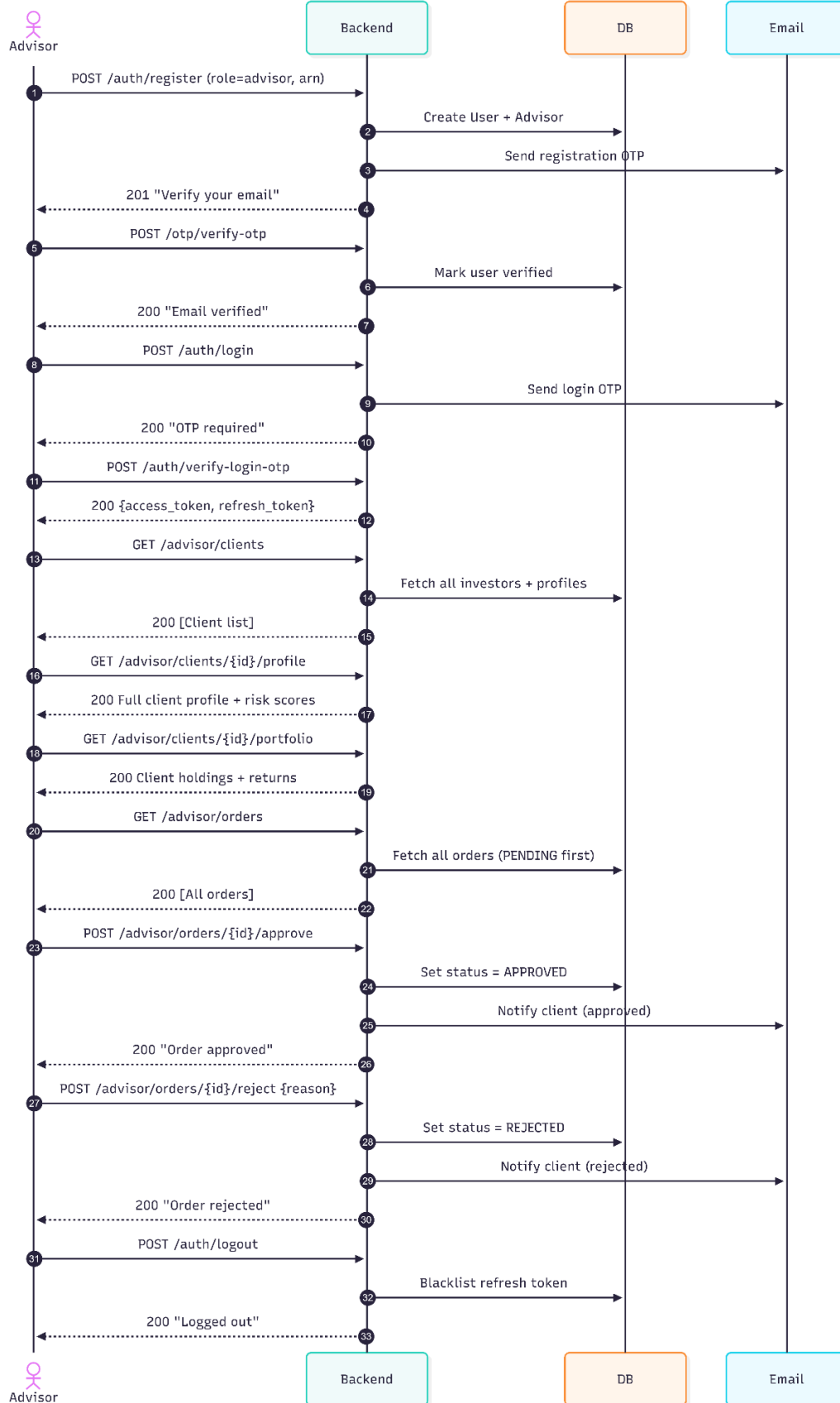
4. Technology Stack

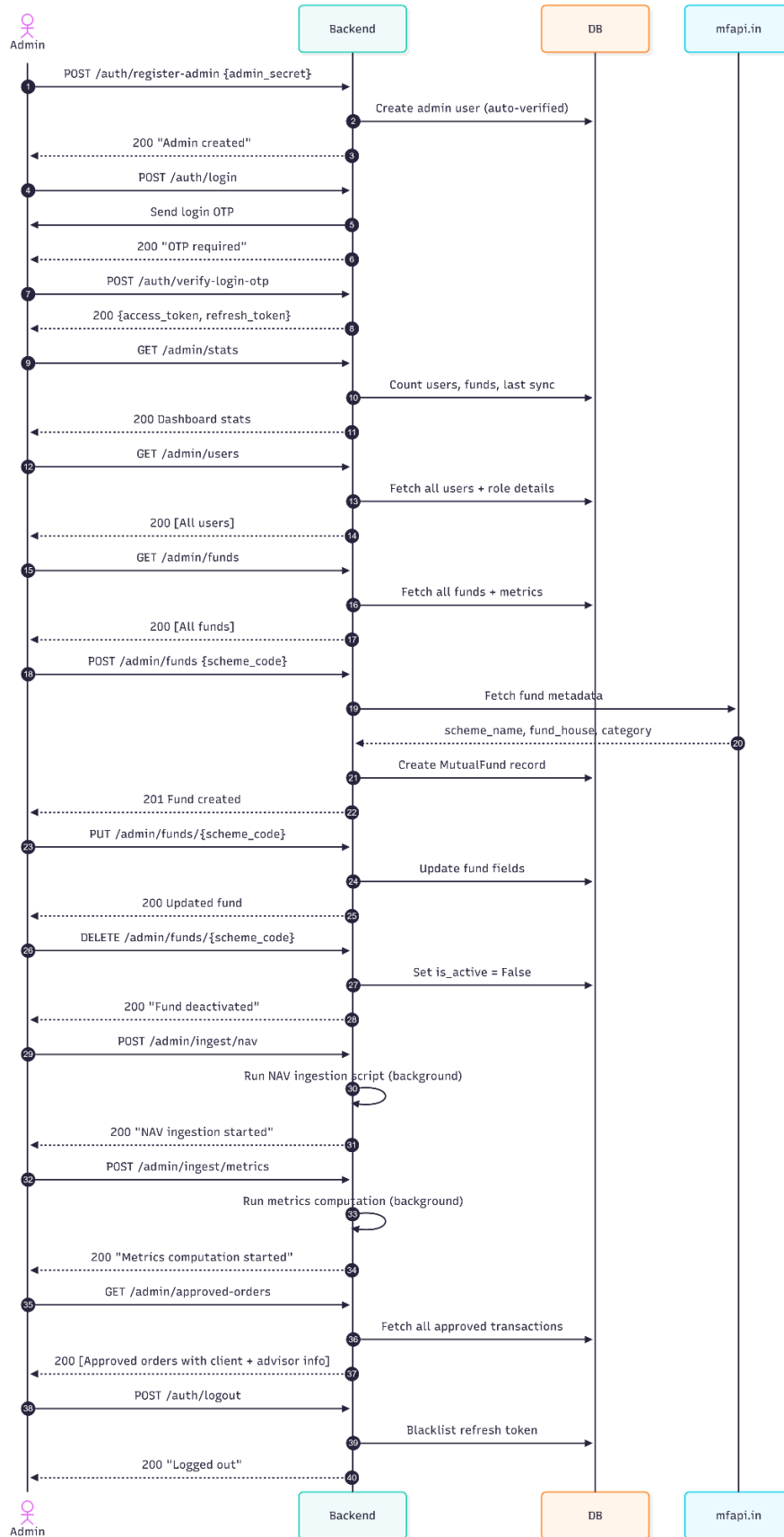
Component	Technology
Frontend	React
Backend	Python FastAPI
API Server	Uvicorn
ORM	SQLAlchemy
Database	PostgreSQL
Authentication	JWT

Component	Technology
Security	Password hashing (bcrypt)
Deployment	Docker
Cloud	AWS
Notifications	Email / SMS Service









5. Presentation Layer Design

The Presentation Layer provides the user interface for interacting with the system.

Technology

React Web Application

Responsibilities

- Collect investment inputs from the user
- Capture investment amount
- Capture investment duration
- Capture optional risk preference
- Display recommended fund allocation
- Display explanation of recommendations
- Display portfolio insights

The frontend communicates with backend APIs using **RESTful JSON requests**.

6. Infrastructure Layer Design

The Infrastructure Layer manages traffic routing and request handling.

Component

AWS Application Load Balancer (ALB)

Responsibilities

- Receive incoming client requests
- Perform **TLS termination**
- Route traffic to backend API servers
- Perform health checks on backend services
- Distribute traffic across backend instances

This layer improves **availability and scalability** of the backend services.

7. API Layer Design

The API Layer is implemented using **FastAPI**.

It acts as the entry point for all client requests.

Responsibilities

- Define REST API endpoints
- Validate request inputs using **Pydantic schemas**
- Authenticate users using **JWT tokens**
- Enforce **Role Based Access Control (RBAC)**
- Route requests to appropriate application modules
- Return structured JSON responses

API Characteristics

- Stateless REST APIs
- Versioned endpoints (`/api/v1`)
- Async request handling
- Secure authentication

8. Application Layer Design

The Application Layer contains business logic modules responsible for coordinating system workflows.

Modules include:

8.1 Authentication Module

Responsibilities:

- User registration
- User login
- Password hashing
- JWT token generation
- Role validation

8.2 Client Profile Module

Responsibilities:

- Store user demographic information
- Store investment preferences
- Retrieve client profile information

8.3 Portfolio Module

Responsibilities:

- Store client mutual fund holdings
- Retrieve portfolio data
- Calculate total portfolio exposure
- Aggregate holdings by category

8.4 Recommendation Module

Responsibilities:

- Invoke the Recommendation Engine
- Store recommendation results
- Retrieve historical recommendations

8.5 Advisor Module

Responsibilities:

- Allow advisors to view client profiles
- Review system-generated recommendations
- Provide advisory notes

9. Recommendation Engine Design

The Recommendation Engine is the core computational component responsible for generating optimized mutual fund allocations.

It processes client data and portfolio information to produce recommendations.

The engine contains several modules.

9.1 Risk Scoring Module

Responsibilities:

- Calculate user risk tolerance
- Normalize risk scores
- Map score to risk category

Inputs:

- Investment duration
- Risk preference
- Portfolio exposure

Output:

User Risk Score

9.2 Portfolio Analyzer

Responsibilities:

- Analyze existing client portfolio
- Calculate portfolio risk exposure

Outputs:

- Portfolio risk profile

9.3 Fund Ranking Engine

Responsibilities:

Evaluate mutual funds based on:

- historical returns
- volatility
- Sharpe ratio
- maximum drawdown
- expense ratio

Output:

Ranked list of candidate funds.

9.4 Allocation Optimizer

Responsibilities:

- Compute final fund score
- Allocate investment proportionally across funds
- Maintain diversification constraints

9.5 Explanation Generator

Responsibilities:

Generate a human-readable explanation for the recommendation.

Example:

- Why each fund was selected
- How the fund matches the user's risk profile
- Why allocation percentages were chosen

10. Repository Layer Design

The Repository Layer isolates database operations from application logic.

Benefits:

- Improves maintainability
- Simplifies testing
- Decouples database access

Repositories include:

UserRepository

Handles:

- user creation
- profile retrieval

PortfolioRepository

Handles:

- storing portfolio holdings
- retrieving portfolio data

FundRepository

Handles:

- retrieving mutual fund data
- retrieving fund metrics

RecommendationRepository

Handles:

- storing recommendation results
- retrieving recommendation history

11. Data Layer Design

The system uses **PostgreSQL** as the primary relational database.

Core Tables

Users

```
users(  
  user_id,  
  name,  
  email,  
  phone no,  
  password_hash,  
  role  
)
```

Portfolios

```
portfolios(  
  portfolio_id,  
  user_id,  
  fund_id,  
  invested_amount  
)
```

Funds

```
funds(  
  fund_id,  
  fund_name,  
  category,
```

```
risk_level  
)
```

Fund Metrics

```
fund_metrics(  
  fund_id,  
  cagr,  
  volatility,  
  sharpe_ratio,  
  max_drawdown,  
  expense_ratio  
)
```

Recommendations

```
recommendations(  
  recommendation_id,  
  user_id,  
  created_at  
)
```

Risk Rules

```
risk_rules(  
  rule_id,  
  risk_category,  
  parameters  
)
```

12. Error Handling Strategy

The backend implements centralized exception handling.

Strategy

- Global exception handlers
- Standardized error responses
- No internal stack traces exposed

Example error response

```
{  
  "status": "error",  
  "message": "Unauthorized action",  
}
```

```
"trace_id": "req-123"  
}
```

13. Security Design

Security mechanisms include:

- JWT-based authentication
- Role-based access control
- Password hashing (bcrypt)
- TLS encryption via Load Balancer
- Input validation using Pydantic
- Secure environment variables

14. Performance & Scalability Design

Performance optimizations include:

- In-memory recommendation computation
- Precomputed fund metrics
- Async FastAPI endpoints
- Indexed PostgreSQL queries

Scalability features include:

- Stateless backend APIs
- Horizontal scaling using containers
- Load balancer traffic distribution

15. Deployment Design

Deployment architecture includes:

- Dockerized backend services
- Environment-based configuration
- AWS infrastructure compatibility

Deployment flow:

React UI
↓
AWS Load Balancer
↓
FastAPI Backend
↓
PostgreSQL Database

16. Sequence Flow – Recommendation Generation

1. User enters investment inputs in UI
2. UI sends request to FastAPI API
3. API validates request using Pydantic
4. Authentication middleware validates JWT
5. Portfolio module retrieves user holdings
6. Recommendation engine runs optimization logic
7. Allocation optimizer produces fund allocation
8. Explanation generator creates explanation text
9. Results stored in database
10. API returns response to UI

17. Assumptions & Constraints

- Historical mutual fund data is used
- Real-time market feeds are not included
- System provides advisory recommendations only
- Trade execution is out of scope

18. Conclusion

This Low Level Design provides a detailed blueprint for implementing the **Mutual Fund Purchase Optimizer System**.

The architecture ensures:

- modular design
- high performance

- explainable recommendations
- scalable cloud deployment

The design aligns with the HLD and supports the development of a production-ready recommendation engine for mutual fund investment optimization.